

Actividad 1 de microcontroladores.
Castañón Salazar Alejandro
30/09/26

Resumen:

Este documento describe cómo se diseñó e implementó un generador de frecuencias usando un ATmega328P, programado en ensamblador desde Microchip Studio. El sistema tiene dos entradas, PD2 y PD3, que sirven para elegir una de cuatro frecuencias de salida: 100 kHz, 500 kHz, 1 MHz o 2 MHz. La señal se saca por PB5, el mismo pin al que está conectado el LED integrado de la placa Arduino UNO. Para seleccionar la frecuencia, el programa lee el registro PIND, aísla los bits que interesan con desplazamientos y una máscara, y después usa una serie de comparaciones en cascada para saltar a la rutina adecuada. La onda cuadrada se genera con retardos por software, calibrados en ciclos de reloj para cada frecuencia. El funcionamiento se comprobó en SimulIDE, midiendo las frecuencias con un frecuencímetro virtual. En resumen, trabajar en ensamblador permite controlar el hardware con mucha precisión y entender mejor la arquitectura AVR, algo que sirve como base para desarrollar sistemas embebidos más complejos.

1.- Introducción:

Los microcontroladores AVR se usan mucho en sistemas embebidos por su arquitectura RISC, su eficiencia energética y lo sencillo que resulta programarlos. El ATmega328P, el mismo que trae la Arduino UNO, es un microcontrolador de 8 bits con un equilibrio muy bueno entre prestaciones y simplicidad, lo que lo vuelve una plataforma ideal para aprender y desarrollar proyectos de electrónica digital. Programar en ensamblador permite acceder directamente a los registros del microcontrolador y controlar con precisión cuánto tarda cada instrucción en ejecutarse. Eso es fundamental cuando la temporización es crítica, como sucede al generar señales periódicas. Esta práctica implementa un generador de frecuencias seleccionable. El objetivo principal es entender cómo funcionan los puertos de entrada y salida, cómo se manejan los registros de dirección y datos, y cómo se generan retardos por software a partir del conteo de ciclos de reloj. El sistema lee dos interruptores conectados a PD2 y PD3 y, según la combinación de sus estados, genera una onda cuadrada de 100 kHz, 500 kHz, 1 MHz o 2 MHz en PB5. La simulación se realizó en SimulIDE y las frecuencias se verificaron con un frecuencímetro virtual. Gracias a eso se pudo comprobar que el diseño funciona correctamente sin necesidad de tener físicamente la tarjeta de desarrollo.

2.- Funcionamiento del sistema.

2.1. Microprocesador:

El ATmega328P es un microcontrolador AVR de 8 bits con arquitectura RISC que ejecuta la mayoría de sus instrucciones en un solo ciclo de reloj, alcanzando hasta 16 millones de instrucciones por segundo cuando opera a una frecuencia de 16 MHz. Cuenta con 32 registros de propósito general, de los cuales se utilizaron dos en particular. El registro R16 se empleó para leer el estado del puerto D y almacenar temporalmente el valor de selección obtenido tras los desplazamientos y la máscara. Por su parte, el registro R17 se utilizó como contador en los bucles de retardo que determinan la duración de cada semiperíodo de la onda generada.

En cuanto a los registros de entrada y salida, se utilizó el registro DDRB, cuya dirección de E/S es 0x04, para configurar la dirección del pin PB5 escribiendo un 1 en el bit correspondiente y habilitándolo así como salida. El registro PINB, con dirección 0x03, se aprovechó de una característica especial del hardware AVR: escribir un 1 en un bit de este registro invierte automáticamente el estado del pin de salida asociado, lo que permitió conmutar el LED de manera eficiente sin necesidad de leer y reescribir el registro PORTB. Para las entradas, se utilizó el registro DDRD, con dirección 0x0A, escribiendo un 0 en los bits 2 y 3 para configurar PD2 y PD3 como entradas. Posteriormente, se escribió un 1 en los bits 2 y 3 del registro PORTD, cuya dirección es 0x0B, con el fin de activar las resistencias pull-up internas. Finalmente, el registro PIND, con dirección 0x09, se leyó para obtener el estado lógico actual de los pines y determinar la selección del usuario.

2.2. Memorias:

El ATmega328P dispone de tres tipos de memoria que cumplen funciones distintas dentro del sistema. La memoria Flash, con una capacidad de 32 KB, es una memoria no volátil donde se almacena el programa compilado en lenguaje ensamblador; es aquí donde reside todo el código que el microcontrolador ejecuta tras un reinicio. La memoria SRAM, con 2 KB de capacidad, es una memoria volátil que se utiliza para almacenar variables y gestionar la pila durante la ejecución del programa; aunque en esta práctica no se declararon variables globales, la SRAM se empleó implícitamente para el contexto de las llamadas y el resguardo de registros. Por último, la memoria EEPROM, con 1 KB de capacidad, es una memoria no volátil destinada a almacenar datos que deben persistir incluso después de apagar el dispositivo; en esta práctica no fue utilizada, pero se reconoce su utilidad en aplicaciones donde se requiere conservar configuraciones o valores de calibración.

2.3. Interfaz de para periféricos:

Para la interfaz con los periféricos se utilizaron los puertos B y D del microcontrolador. El pin PB5 del Puerto B se configuró como salida digital. Este pin tiene la capacidad de suministrar o absorber corriente suficiente para polarizar un LED, y en la tarjeta Arduino UNO se encuentra conectado al LED integrado a través de una resistencia limitadora de corriente, lo que facilita la verificación visual del funcionamiento, aunque a las frecuencias manejadas el parpadeo no sea perceptible al ojo humano. Por su parte, los pines PD2 y PD3 del Puerto D se configuraron como entradas digitales con resistencias pull-up internas activadas. Esta configuración permite conectar interruptores directamente a GND sin necesidad de resistencias externas, de modo que cuando el interruptor está abierto la entrada lee un nivel lógico alto de 5 V, y cuando está cerrado lee un nivel lógico bajo de 0 V.

En cuanto a las características eléctricas relevantes, el ATmega328P opera con un voltaje de alimentación que va de 1.8 V a 5.5 V, aunque en la tarjeta Arduino UNO se utiliza típicamente 5 V. Cada pin de entrada y salida puede suministrar o absorber hasta 40 mA, lo cual es más que suficiente para polarizar un LED. Las resistencias pull-up internas tienen un valor que oscila entre 20 kΩ y 50 kΩ, lo que garantiza un consumo de corriente muy bajo cuando el interruptor está abierto. Estas características hacen que la interfaz sea robusta y sencilla de implementar.

3.- Algoritmos:

El programa se estructuró en varios módulos claramente diferenciados: una sección de configuración inicial, un bucle principal encargado de la lectura de entradas y la selección de frecuencia, y cuatro rutinas independientes de generación de frecuencia, cada una con su

propio bucle de retardo calibrado. A continuación se describe el algoritmo general del sistema.

Algoritmo 1: Generador de frecuencias seleccionable

Datos: Estado de los interruptores en PD2 y PD3.

Resultado: Onda cuadrada en PB5 con frecuencia de 100 kHz, 500 kHz, 1 MHz o 2 MHz.

Inicio.

Configurar PB5 como salida mediante sbi DDRB, 5.

Configurar PD2 y PD3 como entradas mediante cbi DDRD, 2 y cbi DDRD, 3.

Activar resistencias pull-up en PD2 y PD3 mediante sbi PORTD, 2 y sbi PORTD, 3.

1. Inicio.
2. Configurar PB5 como salida mediante sbi DDRB, 5.
3. Configurar PD2 y PD3 como entradas mediante cbi DDRD, 2 y cbi DDRD, 3.
4. Activar resistencias pull-up en PD2 y PD3 mediante sbi PORTD, 2 y sbi PORTD, 3.
5. Bucle principal:
 - a. Leer el estado del puerto D con in r16, PIND.
 - b. Desplazar dos veces a la derecha para mover PD2 y PD3 a las posiciones 0 y 1 mediante lsr r16 y lsr r16.
 - c. Enmascarar con 0x03 para aislar los dos bits mediante andi r16, 0x03.
 - d. Comparar r16 con 0, 1 y 2. Si es 0, saltar a rutina de 100 kHz; si es 1, a 500 kHz; si es 2, a 1 MHz; en caso contrario, a 2 MHz.
6. Rutina de 100 kHz:
 - a. Conmutar PB5 con sbi PINB, 5.
 - b. Retardo de 80 ciclos: cargar 26 en R17, ejecutar bucle de decremento y salto, más dos instrucciones nop.
 - c. Volver al bucle principal.
7. Rutina de 500 kHz:
 - a. Conmutar PB5.
 - b. Retardo de 16 ciclos: cargar 5 en R17, ejecutar bucle, más una instrucción nop.
 - c. Volver al bucle principal.
8. Rutina de 1 MHz:
 - a. Conmutar PB5.
 - b. Retardo de 8 ciclos: cargar 2 en R17, ejecutar bucle, más dos instrucciones nop.
 - c. Volver al bucle principal.
9. Rutina de 2 MHz:
 - a. Conmutar PB5.
 - b. Retardo de 4 ciclos: cargar 1 en R17, ejecutar bucle, más una instrucción nop.
 - c. Volver al bucle principal.
10. Fin.

4.- Análisis de resultados:

La simulación en SimulIDE permitió verificar el funcionamiento del sistema, aunque los resultados obtenidos presentaron diferencias notables respecto a los valores teóricos esperados. Se conectaron dos interruptores a los pines PD2 y PD3 con sus resistencias

pull-up internas activadas, y en el pin PB5 se conectó un frecuencímetro virtual para medir la frecuencia de salida. La tabla 1 muestra las frecuencias medidas para cada combinación de entrada.

Entrada (PD3, PD2)de entrada	Frecuencia seleccionada	Frecuencia medida
00	100 kHz	400 kHz
01	500 kHz	347.8 kHz
10	1 MHz	275.9 kHz
11	2 MHz	87.91 kHz

Los valores obtenidos en la simulación no coinciden con los valores teóricos calculados, y más aún, la tendencia observada es inversa a la esperada. En teoría, la combinación 00 debería producir la frecuencia más baja, 100 kHz, y la combinación 11 la más alta, 2 MHz. Sin embargo, en la simulación la combinación 00 produjo la frecuencia más alta medida, 400 kHz, mientras que la combinación 11 produjo la más baja, 87.91 kHz.

Esta discrepancia puede atribuirse a varias causas relacionadas con la naturaleza de la simulación. En primer lugar, SimulIDE no modela con exactitud los ciclos de reloj reales del ATmega328P, especialmente cuando se ejecutan instrucciones de salto condicional y bucles cortos, que son precisamente la base de los retardos por software empleados en este programa. En segundo lugar, la velocidad de simulación reportada por el entorno puede no corresponder uno a uno con el tiempo real del hardware, lo que introduce un factor de escala que afecta las mediciones del frecuencímetro virtual. En tercer lugar, es posible que el frecuencímetro de SimulIDE mida la frecuencia en función del tiempo de simulación y no del tiempo real, lo que genera lecturas inconsistentes cuando la simulación se acelera o se ralentiza dinámicamente para mantener la fluidez de la interfaz.

A pesar de estas diferencias, la simulación sí permitió verificar que el programa responde a los cambios en los interruptores, es decir, que la lógica de selección funciona y que el sistema genera una señal cuadrada en PB5 cuya frecuencia varía según la combinación de entrada. El LED integrado permaneció visualmente encendido sin parpadeo apreciable, lo cual es coherente con las altas frecuencias manejadas y con el fenómeno de persistencia retiniana.

Es importante señalar que los retardos por software son extremadamente sensibles a la frecuencia exacta del reloj del sistema y al tiempo de ejecución real de cada instrucción. En un entorno de simulación, donde estos parámetros no se reproducen con precisión absoluta, es esperable que los valores medidos difieran de los teóricos. Por lo tanto, la simulación resulta útil para validar la lógica del programa, pero no sustituye la verificación en hardware real, donde el cristal de cuarzo de 16 MHz garantiza una base de tiempo estable y predecible.

5.- Conclusión:

La práctica permitió comprender el funcionamiento interno de un microcontrolador AVR y la importancia de la programación en ensamblador para el control preciso del hardware. Se logró implementar un generador de frecuencias seleccionable utilizando puertos de entrada

y salida, registros de dirección y datos, y resistencias pull-up internas para acondicionar las señales de entrada. La lógica de selección, basada en la lectura del puerto D, el desplazamiento de bits y una estructura de comparación en cascada, funcionó correctamente, como lo demostró la respuesta del sistema ante los cambios en los interruptores.

Sin embargo, la verificación de las frecuencias mediante simulación arrojó valores que no coinciden con los teóricos, e incluso mostraron una tendencia inversa a la esperada. Esto evidenció las limitaciones de SimulIDE para modelar con precisión el tiempo de ejecución de las instrucciones a nivel de ciclo de reloj. Los retardos por software dependen críticamente de la frecuencia exacta del reloj y del número exacto de ciclos que consume cada instrucción, incluyendo las de salto y retorno, que en simulación no se reproducen con fidelidad absoluta. En consecuencia, se concluye que la simulación es una herramienta valiosa para validar la lógica y el flujo del programa, pero no es suficiente para certificar la precisión temporal de un diseño basado en retardos por software. La verificación definitiva debe realizarse en hardware real, donde el cristal de cuarzo de 16 MHz proporciona una base de tiempo confiable.